

AI & Agentic Coding

Navigating the Shift from Copilots to Autonomous Agents

Intro

Security Engineer

Responsibilities

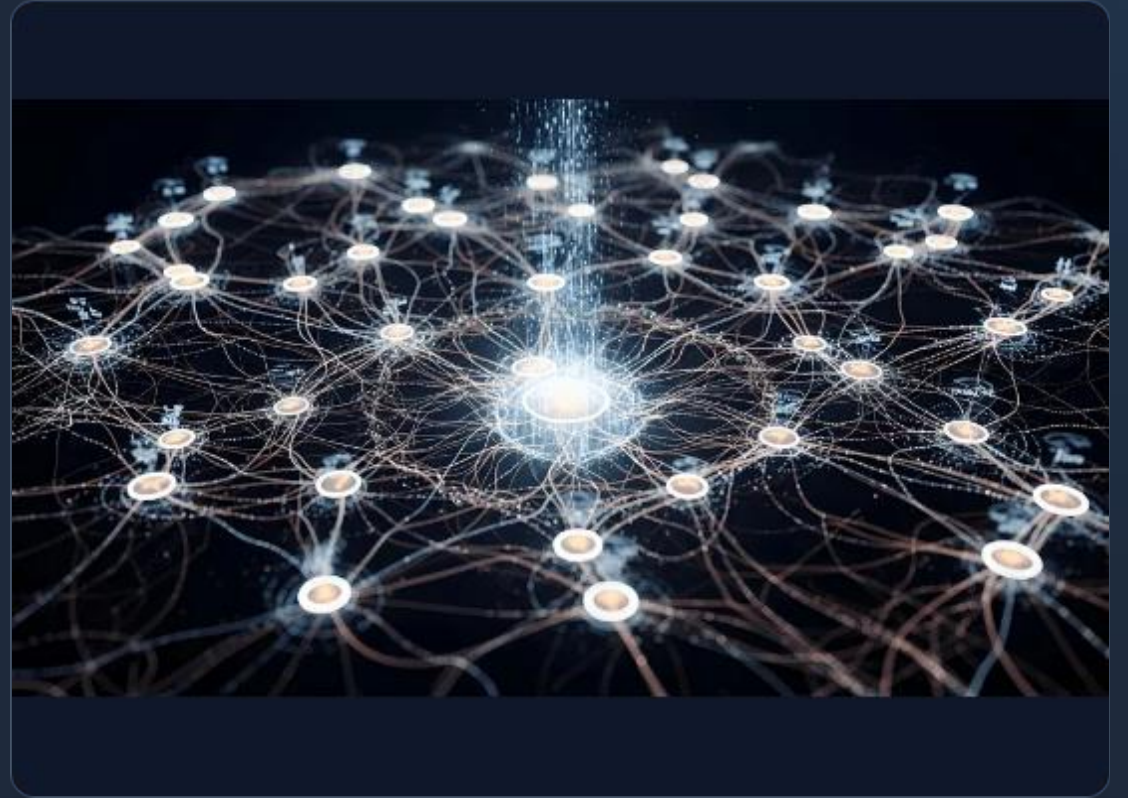
- Consulting
- Find security issues
- Build automation & internal services

The Foundation: AI & LLMs

Large Language Models (LLMs)

Not just text predictors. They are reasoning engines compressed into neural weights.

- Trained on internet-scale code & text.
- Understand syntax, semantics, and intent.
- Capable of zero-shot problem solving.



The Revolution: 2025 Progress



2022

ChatGPT Launch
Basic Code Snippets



2023

GPT-4
Reliable Syntax



2024

Copilot Workspace
Multi-file Edits



2025

Reasoning Models (o3/GPT
5.2)
Autonomous Agents

Current Capabilities



Polyglot Coding

Fluent in Python, JS, Rust, Go, and legacy languages like COBOL. Can translate between them instantly.



Vision & UI

Can see screenshots of bugs or designs and generate the corresponding frontend code.



Tool Use

Can run terminals, read files, browse docs, and execute tests autonomously.

The Model Landscape

Standard Models

(e.g., GPT-4o, Claude 3.5 Sonnet)

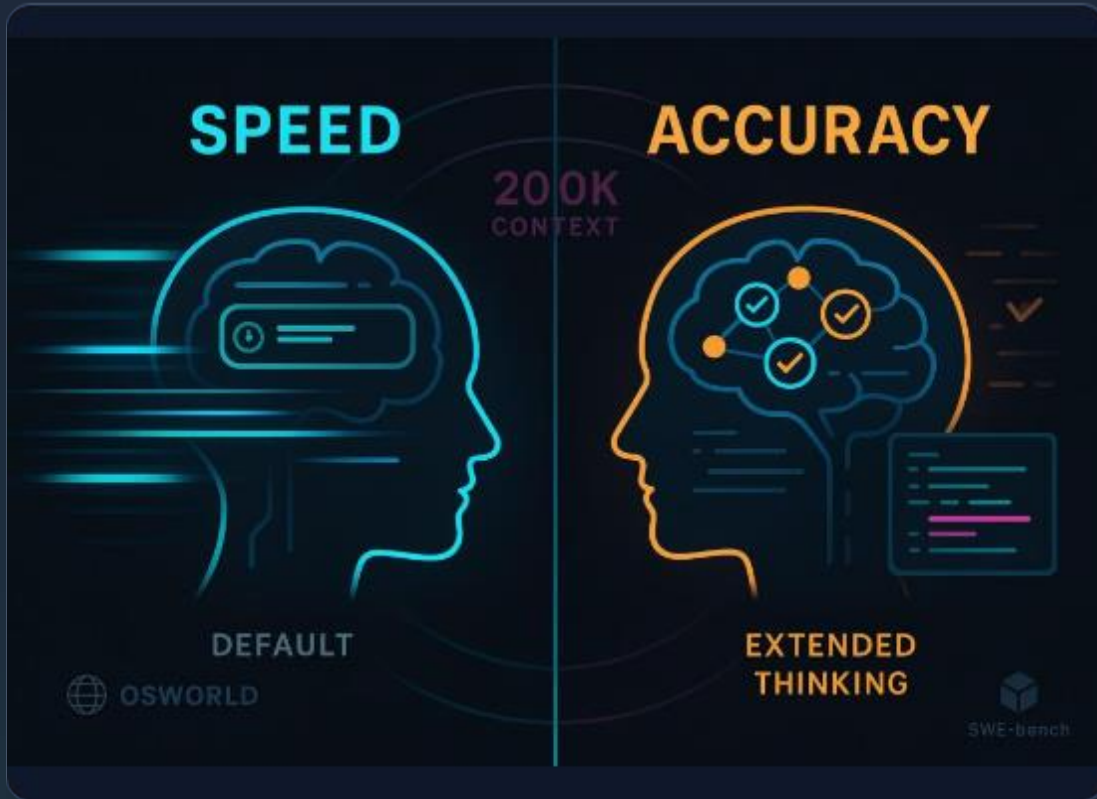
- > Fast & Fluent
- > Great for completion, refactoring, simple tasks.
- > Predicts the next token immediately.
- > "Intuitive" thinking.

Reasoning Models

(e.g., OpenAI o3, DeepSeek R1)

- > **Deep & Deliberate**
- > "Thinks" before outputting tokens.
- > Great for architecture, complex bugs, obscure logic.
- > "Analytical" thinking.

System 1 vs. System 2 Thinking



Matching Task to Model

System 1 (Fast): "Write a regex for email."

-> Use GPT-4o / Haiku. Instant, cheap.

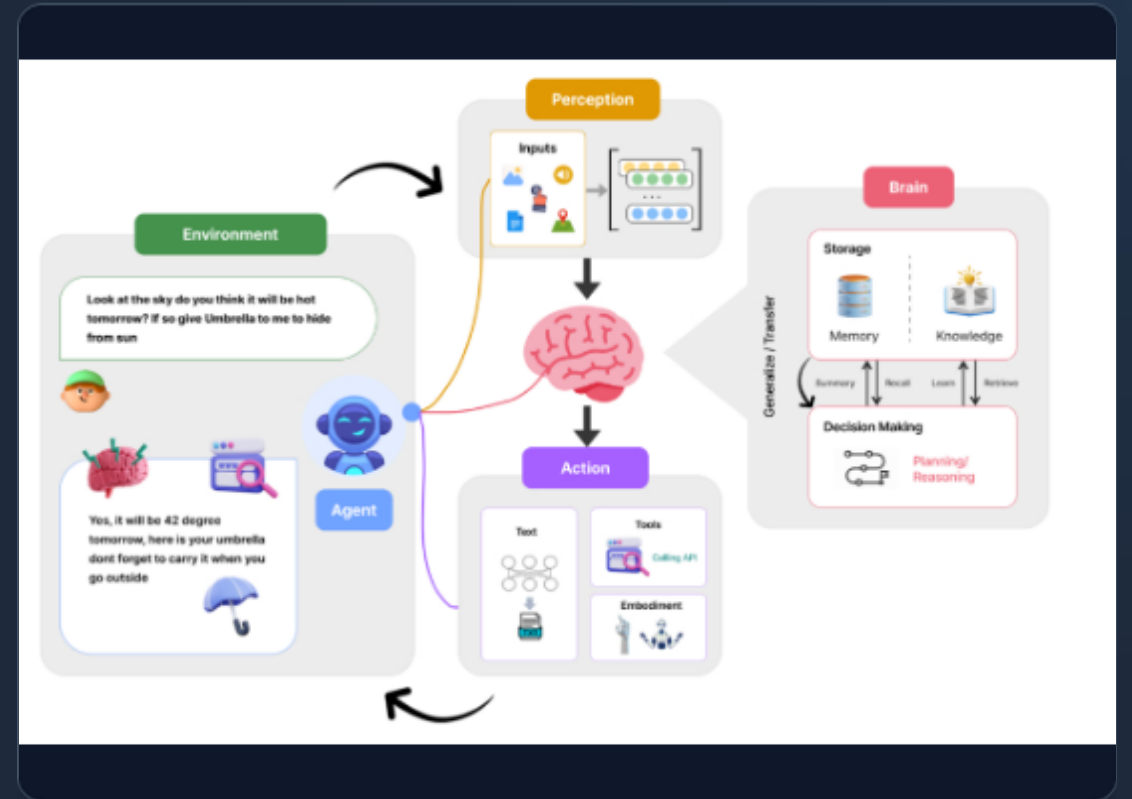
System 2 (Slow): "Why is this race condition happening in my distributed queue?"

-> Use o3 / R1. Expensive, slow, but necessary.

What is an AI Agent?

An LLM is just a brain in a jar. An **Agent** has a body.

- **Perception:** Can read files, see errors, check logs.
- **Brain:** The LLM deciding what to do.
- **Action:** Can write files, run shell commands, call APIs.
- **Loop:** Can try, fail, correct, and retry.



The Evolution of AI Coding

1

Chatbot

Copy/Paste code manually.
Context lost when tab closes.
"Help me write X."

2

Copilot

Autocomplete in IDE. Context aware of open file. "Complete this line."

3

Agent

Autonomous loops. Access to terminal and file system. "Fix this issue and run tests."

Applied AI: The New Stack

- > **IDE Integration:** Cursor, VS Code Copilot.
- > **CLI Agents:** Claude Code, Aider, Open Interpreter.
- > **CI/CD Bots:** CodeRabbit, PR reviewers.

The developer is no longer just a typist, but an **architect and reviewer**.

Prompt Engineering 101

Bad Prompt

"Fix the bug in this function."

Vague, assumes context, no constraints.

Good Prompt

"Act as a senior python engineer. Analyze the ``process_data`` function. It throws a `TypeError` when input is `None`. Fix it by adding a guard clause and add a type hint. Do not change other logic."

Context is King

Models are only as smart as the information you give them.

- **@ Mention Files:** Explicitly add relevant files to context.
- **Paste Errors:** Don't just say "it failed." Paste the stack trace.
- **Reference Docs:** Paste API docs for 3rd party libraries.
- **Project Rules:** Remind it of your linting/style guide.



Advanced Prompting Strategies

- > **Chain of Thought (CoT):** Ask the model to "Think step-by-step" before generating code.
- > **Few-Shot Prompting:** Provide 2-3 examples of "Input -> Desired Output" to set the pattern.
- > **Role Prompting:** "You are a Security Expert. Audit this code for SQL injection."
- > **Specification/Test-based:** "Implement this feature based on the following specs, ensure all tests pass"

The Context Window Advantage

2023: 8k - 32k tokens (A few files).

2025: 200k - 1M tokens (Entire Repositories).

Global Refactoring

Rename a variable across 50 files confidently.

Dependency Upgrades

Update a library and fix breaking changes everywhere.

Onboarding

"Explain how the auth flow works in this entire codebase."

The Tools Landscape

Category	Examples	Best For
AI IDEs	Cursor, Windsurf	Daily driving, "Tab" autocomplete, Composer mode.
CLI Agents	Codex, Claude Code, Aider	Deep changes, terminal ops, "Fix the build".
Extensions	Copilot, Cody	Integration into existing VS Code setup.

Section: Agentic Coding

Moving from "Help me write this" to "Get this done".

The Agentic Loop

- > **1. Plan:** Analyze the request and map out files to touch.
- > **2. Act:** Edit code, run commands.
- > **3. Observe:** Read terminal output (errors, logs).
- > **4. Correct:** If error, self-heal and retry.



Tools: Giving AI Hands



File System

ls, grep, cat, write. Agents can explore your repo structure to find what they need.



Terminal

Run tests (``npm test``), check git status, install packages (``pip install``).



Browser

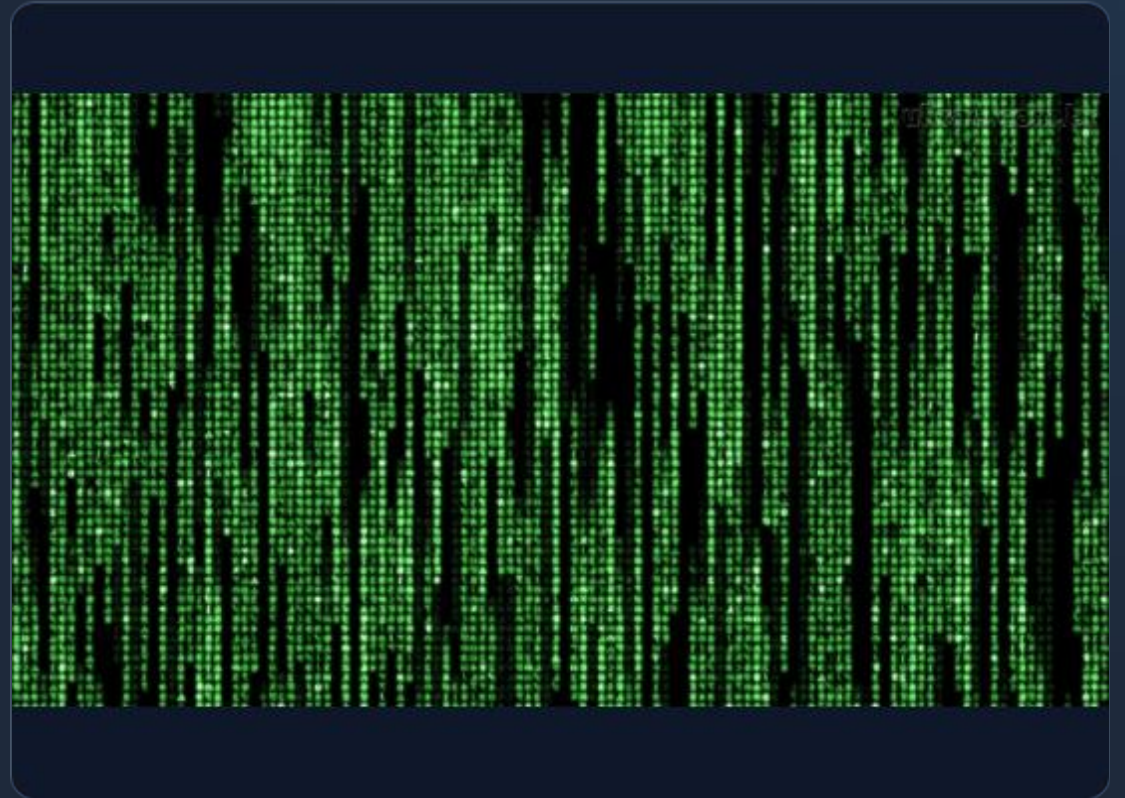
Search documentation, read GitHub issues, verify localhost UI.

CLI Agents (e.g., Codex, Claude Code)

CLI tools bring the agent directly into your terminal.

- > **Authentication:** Securely logs in.
- > **Project Context:** Indexes your local files.
- > **Autonomous:** "Claude, run the test suite and fix any failures."

```
$ claude "Refactor main.py to use async"
```



Best Practice: Plan Mode

The Mistake

Allowing the agent to start editing files immediately without a strategy.

The Solution

Force a "**Plan Phase**".

"Research the codebase and propose a plan. Do not edit yet."

Review the plan -> Approval -> Execute.

Best Practice: Small Diffs

- Agents are prone to "drift" over long tasks.
- **Rule:** Break big features into small, atomic commits.
- **Checkpointing:** Commit working states frequently. If the agent messes up, you can `git checkout` back to safety.

"Make one change, verify it, commit it. Then move to next."

Best Practice: Verification

Never trust the model's output blindly.

Linting

Ask agent to run linter after every edit.

Testing

Write a test case *before* the fix (TDD). Agent must pass the test.

Visual

Ask for a screenshot or describe the UI change.

The Reviewer Role

As AI writes more code, your role shifts to **High-Level Reviewer**.

- Focus on security, architecture, and business logic.
- Ignore syntax (the AI gets that right).
- Spot "Logic Hallucinations" (code that runs but does the wrong thing).



Iterative Prompting Workflow

- > **Draft:** Give a rough instruction.
- > **Review:** See what the AI produced.
- > **Refine:** "Good, but make the variable names more descriptive and handle the edge case where X is null."
- > **Finalize:** Accept and commit.

Use Case 1: Legacy Refactoring

Strategy

Legacy code is scary. AI is fearless.

Use AI to understand spaghetti code, add types, and modularize it.

Need existing test coverage

Prompt

"Explain this `utils.js` file step by step. Then, propose a plan to split it into 3 smaller modules based on functionality. Do not delete any logic, only move it."

Use Case 2: Comprehensive Testing

Strategy

AI excels at generating edge cases you might miss.

Prompt

```
"Analyze `PaymentProcessor.ts`. Write a Jest test suite covering: 1. Happy path. 2. Network timeout. 3. Invalid currency. 4. Zero value transaction."
```

Use Case 3: Documentation

Inline Docs

"Add JSDoc comments to all functions in this file explaining params and return types."

READMEs

"Read the codebase and generate a README.md with Setup, Installation, and Architecture overview."

Commit Messages

"Generate a semantic commit message for these changes."

Use Case 4: Feature Implementation

The "Vertical Slice" Approach

Don't say "Build the dashboard." Say:

```
"1. Create a DB migration for the 'Analytics' table.  
2. Create the backend API endpoint to fetch stats.  
3. Create the React component to display the chart."
```

Guide the agent layer by layer.

Use Case 5: Root Cause Analysis

When you are stuck, paste the error and the code.

Analyze

"Here is the stack trace. Here is the code. What are the top 3 possible causes?"

Fix

"Add logging to the critical path to help us debug this issue."

Example: The "Mega-Prompt"

Role: You are a Senior Frontend Developer.

Task: Create a reusable 'Settings' modal component.

Context: Use **Tailwind CSS** and ensure it matches our existing design system (reference `Button.tsx` for button styles).

Constraints:

- Must be fully accessible using **aria-labels**.
- Must be **mobile-responsive**.
- Use the internal `useModal` hook from `@/hooks`.

Output: Provide only the React component code with inline documentation.

Handling Hallucinations

AI will sometimes invent libraries that don't exist.

- **Symptom:** "Import `super-cool-lib` failed."
- **Fix:** "Are you sure `super-cool-lib` exists? Please use standard `axios` instead."
- **Prevention:** Explicitly tell it which libraries it is allowed to use.

The "Junior Developer" Mental Model

Treat the AI Agent like a very fast, very well-read, but sometimes naive Junior Developer.

- They need clear instructions.
- They need their work reviewed.
- They might miss the "big picture".
- But they work 24/7 and never complain.



AI in Cyber Security

The Double-Edged Sword

AI is transforming security from reactive to predictive.

- **Speed:** Attackers use AI to scale. Defenders use AI to respond.
- **Complexity:** AI agents can now exploit vulnerabilities in complex chains.
- **Adaptation:** Malware that rewrites itself (polymorphic) to evade detection.



Defense: Auto-Remediation



Agentic Patching

Agents don't just alert; they fix.

- **Dependabot on Steroids:** Reads a CVE, finds the library, updates it, fixes breaking changes, and runs tests.
- **Code Scanning:** Identifying buffer overflows or SQL injection patterns and proposing a sanitized rewrite instantly.

The Threat: AI-Powered Offense



Spear Phishing

AI generates hyper-personalized emails using your public LinkedIn data, making scams indistinguishable from real comms.



Bug finding

AI agents autonomously find zero-day exploits faster than any human.



Deepfakes

Voice and video cloning used to bypass biometric security or trick employees into authorizing transfers.

Enterprise Adoption Trends



Private Cloud

Running models on-prem, zero-data retention, or via VPC (OpenAI Enterprise, Azure OpenAI, AWS Bedrock) to protect IP.



RAG on Codebases and internal Knowledge

Indexing internal documentation so AI knows "How we do things here."



Governance

Policies on what code can be sent to AI and attribution.

Enterprise AI Maturity

- **Ad-Hoc (Level 1):** Devs using ChatGPT in secret. Shadow IT risk.
- **Enabled (Level 2):** Official seats for GitHub Copilot. Basic training.
- **Optimized (Level 3):** RAG on internal wikis. Custom instructions. Security scanners integration.
- **Transformative (Level 4):** AI-Native workflows. Agents handle on-call triage and routine maintenance.

How Big Tech Applies AI

Google

Uses AI to rewrite code from C++ to Rust for memory safety. 50% of new code is AI-generated (reviewed by humans).

Meta

"CodeCompose" suggests changes across the monorepo. Automated test generation for billions of lines of code.

And more ...

- OpenAI: Android app developed in 18 days with codex assistance, some repositories are 100% AI-written
- Anthropic: Claude Code (their internal CLI tool) for roughly 60% of their work, reporting a 50% overall productivity boost.
- Microsoft : AI-Generated Code: As of late 2025, confirmed that 20% to 30% of its internal code is being written by AI

Measuring ROI

It's not just "Lines of Code produced".

Velocity

Time from Idea -> PR Merge.

Quality

Reduction in bugs per feature.
Higher test coverage.

Satisfaction

Developer happiness (less boilerplate, more interesting work).

The Future of Work

Coding barriers are lowering.

Software Engineering (including Security) is shifting to
System Engineering.

The skill is defining the **problem**, not implementing the syntax.



Upskilling: What Matters Now?

- **System Design:** Understanding scalability, databases, and arch.
- **AI Literacy:** Knowing how to prompt, fine-tune, and agent-drive.
- **Code Review:** Being able to spot bugs in code you didn't write.
- **Soft Skills:** Requirement gathering and stakeholder management.

Mindset shift: think of yourself as Team Lead/Technical Architect/Manager instead of Individual Contributors

Common Pitfalls

Over-Reliance

Juniors forgetting how to code without AI. "The Pilot who can't fly manual."

Security Leaks

Pasting API keys or PII into public LLM chats.

Building an AI-Native Culture

- Encourage experimentation. Give devs budget for tools.
- Share "Prompt Libraries" internally.
- Host "Hackathons" to automate internal toil.
- Don't punish using AI; reward **effective** use of AI.

Key Takeaways

- > **AI is a Tool, not a Replacement:** It amplifies ability.
- > **Agents are the Next Step:** Move from completion to autonomous loops.
- > **Context is Critical:** Learn to manage what the AI sees.
- > **Review Everything:** You are the Director.

Next Step

- Install an AI IDE (Cursor) or Agent (Codex/Claude Code).
- Pick a boring task (docs, tests) and force yourself to do it 100% with AI.
- Observe where it fails and refine your prompts.
- Share your findings with your team.

Questions?